

ITS65704 DATA SCIENCE PRINCIPLES

GROUP ASSIGNMENT

HAND OUT DATE: July 3, 2025

HAND IN DATE: July 3, 2025, 5:00PM

Instructions to students:

- The group assignment should be attempted in a group of 3-4 members.
- Complete this cover sheet and attach it to your submission – this should be your first page.

Student declaration:	
<i>I declare that:</i> ▪ <i>I understand what is meant by plagiarism.</i> ▪ <i>The implication of plagiarism and usage of AI generative tools have been explained to us by our lecturer.</i> <i>This project is all our work and I have acknowledged any use of the published or unpublished works of other people.</i>	
GROUP 18	
Project Title: Money Laundering Detection	
Name	Student ID
Pavithra A/P Uthaya Kumaran	0359490
Enshin Lee	0356680
Khor Yan Xin	0360707
Nur Aisya Shasmeen Binti Ruzaima Effendi	0356999
Samyutthia Ravi Chenther	0369556

Avoid copy and paste jobs in your report and it is considered plagiarism. Plagiarism in all forms is forbidden. Students who submit plagiarised documents will deserve 0 marks.

Marking Rubric

Criteria	Excellent (30%)	Good (24%)	Satisfactory (18%)	Needs Improvement (12%)	Poor (6%)
Background and Project/Business Goal (Marks:3.0)	Provides a clear, detailed background with a well-defined project goal.	Background and goal are clear but lack some detail.	Background and goal are present but vague.	Background or goal is unclear or incomplete.	Background and goal are missing or irrelevant.
Project Background (Marks:3.0)	Provides comprehensive background information.	Provides sufficient background information.	Background information is basic and lacks depth.	Background information is unclear or incomplete.	Background information is not provided or irrelevant.
Explanation of Project Goal (Marks:3.0)	Clearly defines and explains the project goal.	Adequately defines the project goal.	Project goal is defined but lacks clarity.	Project goal is vague or incomplete.	Project goal is not defined or relevant.
Data Set Description (Marks:3.0)	Provides a thorough description of the dataset, including clear examples.	Provides an adequate description of the dataset.	The dataset description is present but lacks depth.	The dataset description is vague or incomplete.	The dataset description is not provided or unclear.
High-Level Statistics (Marks:3.0)	Presents comprehensive and relevant statistical summaries.	Provides relevant statistical summaries.	Statistical summaries are present but limited.	Statistical summaries are unclear or incomplete.	Statistical summaries are missing or irrelevant.
Data Preprocessing and Issues (Marks:3.0)	Thoroughly describes data issues and presents well-executed preprocessing steps.	Adequately describes data issues and preprocessing steps.	Data issues and preprocessing steps are present but basic.	Data issues or preprocessing steps are vague or incomplete.	Data issues and preprocessing steps are not provided.
Data Science Techniques (Marks:3.0)	Explains the techniques used with detailed rationale for each choice.	Explain the techniques used with an adequate rationale.	Techniques explanation is present but lacks detail.	Explanation or rationale of techniques is unclear or incomplete.	Techniques explanation is not provided or relevant.
Model Validation (Marks:3.0)	Clearly explains the model validation process with relevant methods.	Adequately explains the model validation process.	Explanation of model validation is present but basic.	Explanation is unclear or incomplete.	Model validation process is not explained.
Conclusion (Marks:3.0)	Provides insightful observations and comprehensive suggestions.	Provides adequate observations and suggestions.	Observations and suggestions are basic.	Observations or suggestions are vague or incomplete.	Observations and suggestions are missing or irrelevant.
Google Colab (Marks:3.0)	Attach your Colab file for review. Make sure the file does not have restricted access.	Attach your Colab file for review.	Colab file is attached but may have limited access or issues.	Colab file is attached but access is restricted or there are major issues.	No Colab file is attached, or access is completely restricted.

Table of Content

Executive Summary	4
1.0 Background and Project/Business Goal	5
2.0 Project Background	6
3.0 Explanation of Project Goal	8
4.0 Data Set Description	9
5.0 High Level Statistics	13
5.1 Statistical Summary	13
5.2 Exploratory Data Analysis (EDA)	17
6.0 Data Pre-processing and Issues	23
6.1 Random Sampling	23
6.2 Time-Based Feature Engineering	24
6.3 Handling Skewed Data	26
6.4 One-Hot Encoding	27
7.0 Data Science Techniques	29
7.1 XGBoost	29
7.2 Random Forest	33
8.0 Model Validation	35
8.1 XGBoost	35
8.2 Random Forest	40
9.0 Conclusion	43
10.0 Colab Notebook	44
11.0 References	45

Executive Summary

This research explores using machine learning to combat money laundering by developing a predictive classification model trained on the SAML-D dataset. Illegal financial flows are estimated at 2-5% of world GDP annually, and traditional rule-based Anti-Money Laundering (AML) systems are inadequate for detecting complex patterns of money laundering among high levels of transaction. This project used a dataset which had around 9.5 million records of transactions, where there were no missing or duplicate values among the records. Class imbalances in the dataset were addressed through random sampling (using 200,000 records) and SMOTE resampling. Preprocessing included time-based feature engineering, log transformation to address skewness, and one-hot encoding. Exploratory Data Analysis (EDA) on the dataset captured common behaviors of money laundering, such as cross-border transactions, and frequent flagged receiver accounts. Two models of machine learning, Random Forest and XGBoost, were trained and validated. Using validation techniques, the XGBoost model showed that it had a better performance, with a 97% accuracy and an AUC value of 0.99. This model outperformed the Random Forest model, which had a 95% accuracy and AUC value of 0.98. The findings in the research show that XGBoost, is capable of significantly enhancing AML systems by reducing false positives and increasing the efficiency of detecting 'normal' or 'suspicious' transactions. This research contributes to the development of intelligent fraud detection in financial systems and also improves the integration of data science in the security of global finance in the current digital era.

1.0 Background and Project/Business Goal

Money laundering is a serious international problem, with 2-5% of worldwide GDP (\$800 billion to \$2 trillion) being laundered every year (United Nations, 2024). For example, in the UK Fowler Oldfield scandal, \$266 million was laundered through a scrappage jewellery business in Bradford between 2014 and 2016 that is now closed down. The operation is known as the biggest money laundering case in UK history. There were daily cash deposits of up to \$1.7 million from cities including Manchester and Glasgow. Money was put into banks and invested in gold grain, making it difficult to trace. Four individuals were found guilty, and NatWest Bank was fined \$265 million in 2021 for not identifying and reporting the suspicious \$365 million of money (O'Donoghue, 2025).

From the recent news, the UK Financial Conduct Authority (FCA) is identifying unregistered cryptocurrency ATMs. These devices enable individuals to anonymously exchange money for electronic money, making them a desirable product for criminal finance. These activities, according to the FCA, are designed to "prevent illegal activity" and cause an emerging threat to the digital financial marketplace (Schickler, 2023). These actions are usually undetected due to the large daily volume of transactions and the complicated money laundering strategies.

In spite of the evolution of anti-money laundering (AML) technology, the majority of systems are poor at identifying frauds successfully because they lack rich and varied data. In the view of the FATF (Financial Action Task Force), there are few suspicious activity reports that are trapped, and most current AML systems are overwhelmed by the volume of data and are not technologically advanced enough to identify faint patterns of fraud (Flagright, 2024).

To address these issues, this project aims to use data science techniques to overcome such challenges by detecting suspicious financial transactions. This is to enhance existing AML models to provide more efficient tools for the detection and prevention of money laundering operations. Thus, this will have a positive influence on the overall improvement of global financial security.

2.0 Project Background

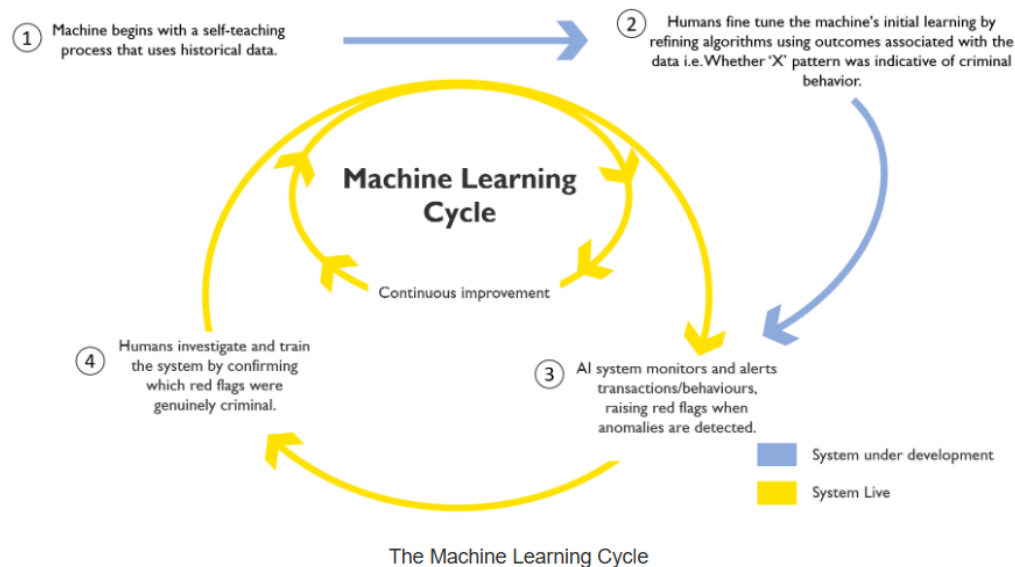


Figure 2.0.1: The machine learning cycle (Napier AI, 2020)

Money laundering is the process of obtaining or transferring money made from illegitimate activities without jeopardizing their sources (United Nations, 2024). Money laundering begins with the movement of the money earned through criminal activity to one or more legitimate financial networks or institutions where layering occurs which is continuing transfer of the money through multiple methods to make it impossible to trace the money back to the illegal origin. The process ends with the final integration where the money is accessible legally and it doesn't require any concealment. Major financial institutions such as banks are usually taken advantage of during the money laundering process as a prestigious bank such as HSBC have been found guilty in aiding in money laundering activity due to not properly reporting large deposits of cash and was found to facilitate the laundering of almost \$1 billion in 2012 (CFI Team, 2015).

Incorporating machine learning algorithms into AML provides various benefits which includes improved detection accuracy as machine learning enables us in identifying complex patterns and anomalies within the large dataset by continuously analyzing data. Apart from that, identifying potential risks, streamlining operations in financial institutions and allocating resources more

effectively are enabled through machine learning as it automates the process of monitoring and detection of suspicious activities which allows continuous and real-time analysis of transactions (Napier AI, 2020). False positives alerts is one of the common challenges identified in the current AML compliance and machine learning algorithms has the capability to assist in reducing the false positives by analyzing and classifying data precisely, resulting in a more efficient and focused investigation procedure (Financial Crime Academy, 2024). Last but not least, AML has the need to adapt to the constantly evolving money laundering tactics and machine learning has the flexibility to learn and adapt that is required for the AML, enabling the detection of emerging patterns and keeping tread with challenging techniques (Napier AI, 2020).

The problem statement is that the current fraud detection systems in digital financial services struggle to keep up with increasingly sophisticated money laundering techniques, necessitating more adaptive and intelligent solutions. Therefore, implementing more adaptive and intelligent fraud detection methods, such as those powered by machine learning models, will be the focus of this project.

3.0 Explanation of Project Goal

The main objectives of this project are

- I. To preprocess the raw transaction data and standardize their values and handle any class imbalances to prepare it for training a machine learning model.
- II. To perform Exploratory Data Analysis (EDA) to identify significant patterns that can differentiate normal transactions from suspicious ones.
- III. To train and evaluate two supervised machine learning models, which are XGBoost and Random Forest that can accurately flag laundering and non-laundering transactions.

The overall goal of the study is to transition Anti-Money Laundering (AML) systems from rigid rule-based filters to more adaptive, data-driven detection mechanisms, and hence increasing the speed of detecting illegal activity and decreasing the workload of manually monitoring the large volume of daily transactions.

4.0 Data Set Description

The money laundering dataset, SAML-D, stands for Synthetic Anti-Money Laundering Dataset which simulates real-life transactions and is sourced from Kaggle. This dataset was created for a study conducted by B. Oztas et al. to help researchers when training and validating their models, as well as help them develop better methods to monitor transactions (Oztas et al., 2023) as using actual real-life transactions will contradict policy regulations and breach individual account owners' privacy.. The dataset has a variety of attributes that are related to money laundering transactions, such as the currency the money is sent and received, the amount of money received and the accounts of the sender and receiver of the money. There are a total of 12 attributes in the dataset, and a total of 9504852 records. The dataset is found to have no duplicated records and no missing values for any of the attributes.

```
print("First 5 rows of the dataset:\n", df.head())
```

First 5 rows of the dataset:

	Time	Date	Sender_account	Receiver_account	Amount	\
0	10:35:19	2022-10-07	8724731955	2769355426	1459.15	
1	10:35:20	2022-10-07	1491989064	8401255335	6019.64	
2	10:35:20	2022-10-07	287305149	4404767002	14328.44	
3	10:35:21	2022-10-07	5376652437	9600420220	11895.00	
4	10:35:21	2022-10-07	9614186178	3803336972	115.25	

	Payment_currency	Received_currency	Sender_bank_location	\
0	UK pounds	UK pounds	UK	
1	UK pounds	Dirham	UK	
2	UK pounds	UK pounds	UK	
3	UK pounds	UK pounds	UK	
4	UK pounds	UK pounds	UK	

	Receiver_bank_location	Payment_type	Is_laundering	Laundering_type
0	UK	Cash Deposit	0	Normal_Cash_Deposits
1	UAE	Cross-border	0	Normal_Fan_Out
2	UK	Cheque	0	Normal_Small_Fan_Out
3	UK	ACH	0	Normal_Fan_In
4	UK	Cash Deposit	0	Normal_Cash_Deposits

Figure 4.0.1: First 5 rows of the dataset

```
# Shape of dataset
print("Shape of the dataset:\n", df.shape)
```

Shape of the dataset:
(9504852, 12)

Figure 4.0.2: Shape of the dataset

The output of Figure 4.0.2 shows that the dataset has 12 attributes and 9,504,852 records.

```
[ ] # Check duplicate records:
    duplicates = df.duplicated()
    num_duplicates = duplicates.sum()
    print(f"Numbers of duplicated rows: {num_duplicates}")
```

➞ Numbers of duplicated rows: 0

Figure 4.0.3: Check for duplicate records

The output in Figure 4.0.3 shows that there are no duplicate records in the dataset, so no further preprocessing steps are needed to remove any duplicates. Each transaction is unique, making it possible to uncover insights from varied transactions.

```
# Find range of date
df['Date'] = pd.to_datetime(df['Date'])

# If Date column is of the type object otherwise skip this ^
date_range = str(df['Date'].dt.date.min()) + ' to ' + str(df['Date'].dt.date.max())
print(date_range)
```

2022-10-07 to 2023-08-23

Figure 4.0.4: Find the ranges of date

The minimum and maximum date values span roughly 1 year, from October 2022 to August 2023. This ensures that the data captures seasonal and regulatory changes that can influence money laundering behaviour over time.

```
# Find min and max of time
print(df['Time'].min())
print(df['Time'].max())

# Find min and max of money sent
print(df['Amount'].max())
print(df['Amount'].min())
```

00:00:00
23:59:59
12618498.4
3.73

Figure 4.0.5: Check for the min and max of time and amount of money

The range of the transaction times covers the full 24-hour cycle, and the amount of money in transactions ranges from very small to very large values. These extremes suggest the need for scaling or logarithmic transforms to stabilise variance before modelling.

```
# Find number of Unique Values for each columns
print(df.nunique())
```

Time	86400
Date	321
Sender_account	292715
Receiver_account	652266
Amount	2314277
Payment_currency	13
Received_currency	13
Sender_bank_location	18
Receiver_bank_location	18
Payment_type	7
Is_laundering	2
Laundering_type	28

dtype: int64

Figure 4.0.6: Number of unique values for each column

Some columns contain many different values. For example, *Receiver_account* shows 652266 distinct accounts and *Amount* has over 2.3 million different values. Other features have only a few possible values, such as *Is_laundering* (2 values) or *Payment_type* (7 values). Identifying the columns that have a high or low cardinality helps determine which suitable encoding method to use and prevent the dataset from becoming too complex or memory-intensive when training the model.

```
# Find missing values
print("Missing values:\n", df.isnull().sum())
```

Missing values:	
Time	0
Date	0
Sender_account	0
Receiver_account	0
Amount	0
Payment_currency	0
Received_currency	0
Sender_bank_location	0
Receiver_bank_location	0
Payment_type	0
Is_laundering	0
Laundering_type	0

dtype: int64

Figure 4.0.7: Number of missing values for each column

Figure 4.0.7 shows that none of the columns contain missing values. This allows the data to be used directly in training the machine learning model without any additional imputation steps needed.

Table 4.0.1: Attributes of the Dataset

Column Name	Description	Data Type
Time	The time of the transaction	object
Date	The date of the transaction	object
Sender_account	ID number of the account transferring the money	int
Receiver_account	ID number of the account receiving money	int
Amount	Amount transferred in the transaction	float
Payment_currency	Currency that the money was transferred	object
Received_currency	Currency that the money was received	object
Sender_bank_location	Location of the bank account sending the money	object
Receiver_bank_location	Location of the bank account receiving the money	object
Payment_type	The type of payment made	object
Is_laundering	If the transaction was money laundering	int
Laundering_type	The type of money laundering activity	object

5.0 High Level Statistics

This section will show the high-level overview of the chosen dataset, SAML-D in terms of the statistical summary and Exploratory Data Analysis (EDA) to gain deeper insights about the dataset for data modelling and decision-making.

5.1 Statistical Summary

This section details the statistical overview of the SAML-D dataset which contains 9.5 million recorded financial transactions. The statistical summary will include the structure of the dataset, the central tendencies in the dataset, categorical and numerical variable distributions as well as the distribution of records between normal and flagged money laundering transactions.

```
#Statistical summary

#1. Data Info
print(df.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9504852 entries, 0 to 9504851
Data columns (total 12 columns):
#   Column                                Dtype
---  -
0   Time                                  object
1   Date                                 datetime64[ns]
2   Sender_account                       int64
3   Receiver_account                     int64
4   Amount                              float64
5   Payment_currency                     object
6   Received_currency                    object
7   Sender_bank_location                 object
8   Receiver_bank_location               object
9   Payment_type                         object
10  Is_laundering                        int64
11  Laundering_type                      object
dtypes: datetime64[ns](1), float64(1), int64(3), object(7)
```

Figure 5.1.1: Data Structure Overview

The `df.info()` function in Figure 5.1.1 enables checking of the dataset structure. The output shows that the dataset contains 9,504,852 rows and 12 columns which reflects the real life volume of financial transactions data. It shows that the date column is of the `datetime64[ns]` type while numerical columns include `Sender_account`, `Receiver_account`, `Amount`, and `Is_laundering`, and categorical columns include `Payment_currency`, `Received_currency`, `Payment_type`, `Sender_bank_location`, `Receiver_bank_location`, `Laundering_type`, and `Time`. The variety in the types of data is useful for EDA in the upcoming section.

```
#2. Check class distribution
print()
df['Is_laundering'].value_counts(normalize=True)
# Note here - <1% flagged transactions (imbalance), to address before model training
```

	proportion
Is_laundering	
0	0.998961
1	0.001039

dtype: float64

Figure 5.1.2: Class Distribution

This code calculates the proportion of the 2 classes in the Is_laundering column which is the target column of this dataset. The output shows important information that the dataset is highly imbalanced where class 0 (non-laundering) accounts for 99.9% of the dataset rows while class 1 (laundering) is far less at only 0.1%. This is true in real-life Money Laundering cases as they are a very small percentage in the large volume of daily financial transactions between billions of users. Hence, this imbalance will be addressed in Section 6.0 Data Preprocessing and Issues.

```
#3. Descriptive Statistics
print()
print(df.describe())
```

	Date	Sender_account	Receiver_account	\
count	9504852	9.504852e+06	9.504852e+06	
mean	2023-03-15 21:03:37.048723968	5.006619e+09	5.006006e+09	
min	2022-10-07 00:00:00	9.018000e+03	9.018000e+03	
25%	2022-12-27 00:00:00	2.513133e+09	2.513219e+09	
50%	2023-03-15 00:00:00	5.001017e+09	5.002572e+09	
75%	2023-06-03 00:00:00	7.505051e+09	7.502397e+09	
max	2023-08-23 00:00:00	9.999987e+09	9.999971e+09	
std	NaN	2.885814e+09	2.884763e+09	

	Amount	Is_laundering
count	9.504852e+06	9.504852e+06
mean	8.762968e+03	1.038733e-03
min	3.730000e+00	0.000000e+00
25%	2.143688e+03	0.000000e+00
50%	6.113720e+03	0.000000e+00
75%	1.045846e+04	0.000000e+00
max	1.261850e+07	1.000000e+00
std	2.561495e+04	3.221263e-02

Figure 5.1.3: Descriptive Statistics (Numerical Columns)

The `df.describe()` function gives the statistical summaries for the numerical variables in the dataset which includes the count, mean, std, min, max and quartiles. Based on the output, some of the important highlights include the amount column where the standard deviation value is high at 25,615 when the mean is at 8763. This informs that the amount column is heavily skewed to the right because of the presence of outlier transactions (laundering cases). The Sender and Receiver accounts only act as the account identifier so they are not aggregated meaningfully, but the value range indicates that there is a large and diverse number of accounts. The mean for `Is_laundering` column confirms that the laundering cases only account for 0.1% of the records.

#4. Categorical Summary

```
for col in df.select_dtypes(include='object'):
    print(f"\nTop categories in {col}:\n{df[col].value_counts()}")
```

Top categories in Time:

Time	
15:22:42	201
14:09:05	199
11:35:00	199
14:49:56	198
14:51:59	196
...	
04:04:46	14
03:38:32	14
03:03:05	13
02:29:43	13
03:39:29	12

Name: count, Length: 86400, dtype: int64

Top categories in Payment_currency:

Payment_currency	
UK pounds	9099293
Euro	117164
Turkish lira	27996
Swiss franc	27492
Dirham	27263
Pakistani rupee	27196
Naira	27143
US dollar	26061
Yen	25562
Moroccan dirham	25395
Mexican Peso	24852
Albanian lek	24778
Indian rupee	24657

Name: count, dtype: int64

Top categories in Received_currency:

Received_currency	
UK pounds	8783655
Euro	231911
Pakistani rupee	45993
Yen	45814
Moroccan dirham	45748
Albanian lek	45736
Mexican Peso	45255
Naira	45046
Indian rupee	43757
US dollar	43664
Swiss franc	42931
Dirham	42797
Turkish lira	42545

Name: count, dtype: int64

Top categories in Sender_bank_location:

Sender_bank_location	
UK	9183088
Turkey	20902
Switzerland	20503
Pakistan	20346
UAE	20081
Nigeria	20027
Spain	19391
Germany	19259
USA	19027
Italy	18895
France	18702
Japan	18468
Morocco	18437
Austria	18050
Mexico	17662
Albania	17648
India	17596
Netherlands	16770

Name: count, dtype: int64

Top categories in Receiver_bank_location:

Receiver_bank_location	
UK	8864634
Pakistan	38980
Austria	38844
Albania	38783
Japan	38729
Morocco	38704
Germany	38540
France	38512
Mexico	38385
Nigeria	38272
Netherlands	37314
Spain	37288
India	36936
USA	36850
Italy	36517
Switzerland	36081
UAE	35897
Turkey	35586

Name: count, dtype: int64

Top categories in Laundering_type:

Laundering_type	
Normal_Small_Fan_Out	3477717
Normal_Fan_Out	2302220
Normal_Fan_In	2104285
Normal_Group	528351
Normal_Cash-Withdrawal	305031
Normal_Cash_Deposits	223801
Normal_Periodical	210526
Normal_Plus_Mutual	155041
Normal_Mutual	125335
Normal_Foward	42031
Normal_single_large	20641
Structuring	1870
Cash-Withdrawal	1334
Deposit-Send	945
Smurfing	932
Layered_Fan_In	656
Layered_Fan_Out	529
Stacked Bipartite	506
Behavioural_Change_1	394
Bipartite	383
Cycle	382
Fan_In	364
Gather-Scatter	354
Behavioural_Change_2	345
Scatter-Gather	338
Single_large	250
Fan_Out	237
Over-Invoicing	54

Name: count, dtype: int64

```

Top categories in Payment_type:
Payment_type
Credit card      2012909
Debit card       2012103
Cheque           2011419
ACH              2008807
Cross-border     933931
Cash Withdrawal  300477
Cash Deposit     225206
Name: count, dtype: int64

```

Figure 5.1.4: Categorical Feature Summary

The loop at Figure 5.1.4 counts the number of unique values per categorical column and their frequencies to show the dominant categories. As seen in the output, some important and repetitive patterns can be observed here. UK Pounds is the most used currency type in the transactions followed by Euro and Turkish lira. Similarly, UK Pounds and Euro dominate the most commonly received currency but more diverse currencies like the Pakistani Rupee and Moroccan Dirham can be noted appearing often in the list. In terms of the sender and receiver bank locations, it can be noted that most transactions occur within the UK. There are 4 main types of payment methods used by the account owners which are credit card, debit card, cheque and ACH where cash and cross-border methods are less preferred. For the target column which is the laundering type, most transactions fall under Normal types such as Normal_Fan_Out and Normal_Group which reflect the class distribution observed previously.

```

#5. Grouped Statistics
print(df.groupby('Is_laundering')['Amount'].describe())

```

	count	mean	std	min	25%	75%	max
Is_laundering							
0	9494979.0	8729.875874	21750.032069	3.73	2142.93	6114.63	10458.950000
1	9873.0	40587.666906	419181.131445	15.82	2723.79	5322.79	9789.673317

Figure 5.1.5: Grouping of Amount by Class

The code in Figure 5.1.5 above shows the statistics for the transaction amount which are grouped by their laundering statuses. The output shows that the mean amount for money laundering cases

(class 1) is approximately 5x higher than the normal cases (class 0). It also has a very high maximum amount of 12 million compared to the normal transactions which is only at 1 million. There is a high standard deviation for money laundering transactions which helps to detect sudden transfer of large funds that can be flagged for suspicious activity detection.

5.2 Exploratory Data Analysis (EDA)

This section will explore the dataset to show important patterns and trends to gain insights about the chosen dataset. 5 key questions are used to guide the process of visualisation to derive insights which are shown below.

1. What are the most flagged payment types?
2. What are the most common types of money laundering in the suspicious transactions?
3. Do suspicious transactions involve larger amounts?
4. Are suspicious transactions observed more frequently between certain receivers?
5. How has the volume of flagged transactions changed over time?

```
#1. What are the most flagged payment types?

suspect_counts = df[df['Is_laundering'] == 1]['Payment_type'].value_counts()

suspect_counts.plot(kind='bar', figsize=(10,6))
plt.title('Suspicious Transactions by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```

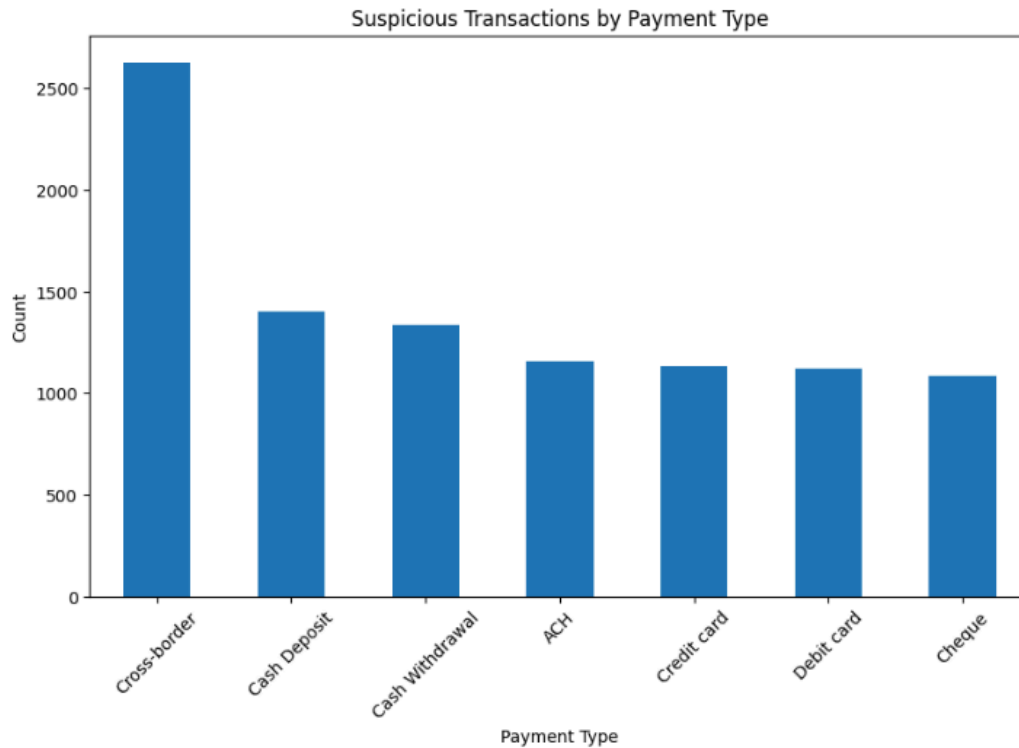


Figure 5.2.1: EDA of the Most Flagged Payment Type

Firstly, the project aims to analyse what are the most flagged payment types in the dataset. The code shown in Figure 5.2.1 filters the transactions (rows) that are labelled as class 1 (laundering) and then counts the number of times each payment type appears. This is then visualised in the bar chart as shown in the figure. The output bar chart shows us that cross-border payment type is the most flagged payment type and it indicates that money laundering cases typically involve transactions across country borders. This calls for special focus in monitoring money flowing in and out of the UK to detect suspicious transactions. The following types are cash methods including deposit and withdrawal which might be done as an attempt to prevent tracking of online transactions.

```
#2. What are the most common types of money laundering in the suspicious transactions?

suspect_laundering = df[df['Is_laundering'] == 1]['Laundering_type']
top_laundering = suspect_laundering.value_counts().head(10)

plt.figure(figsize=(8,8))
top_laundering.plot(kind='pie', autopct='%1.1f%%', startangle=140, colors=plt.cm.Purples(range(50, 250, 20)))
plt.title('Most Common Laundering Types')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

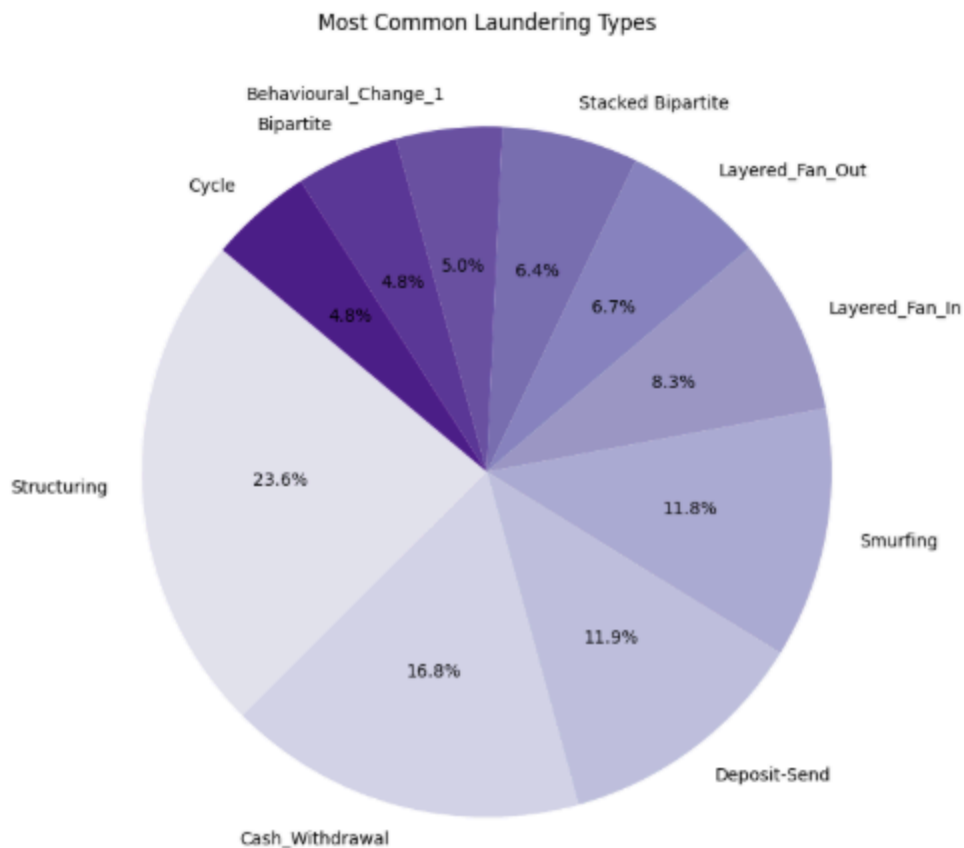


Figure 5.2.2: EDA of the Most Common Laundering Types

Secondly, the code focuses on finding the top 10 most common laundering types among the flagged transactions and using a pie chart to visually show their proportion. As seen in Figure 5.2.2, the output pattern shows that the top money laundering types are structuring which accounts for 23.6%, cash withdrawal and deposit send. Structuring refers to a tactic where large transactions are broken down into smaller and seemingly less suspicious ones to bypass the

thresholds set for automatic flagging. Cash withdrawal is physically withdrawing the funds while deposit-send circulates or layers the fund movement between different entities in order to cover or mask where the amount originates from. These common strategies have clear patterns that can be monitored by financial authorities.

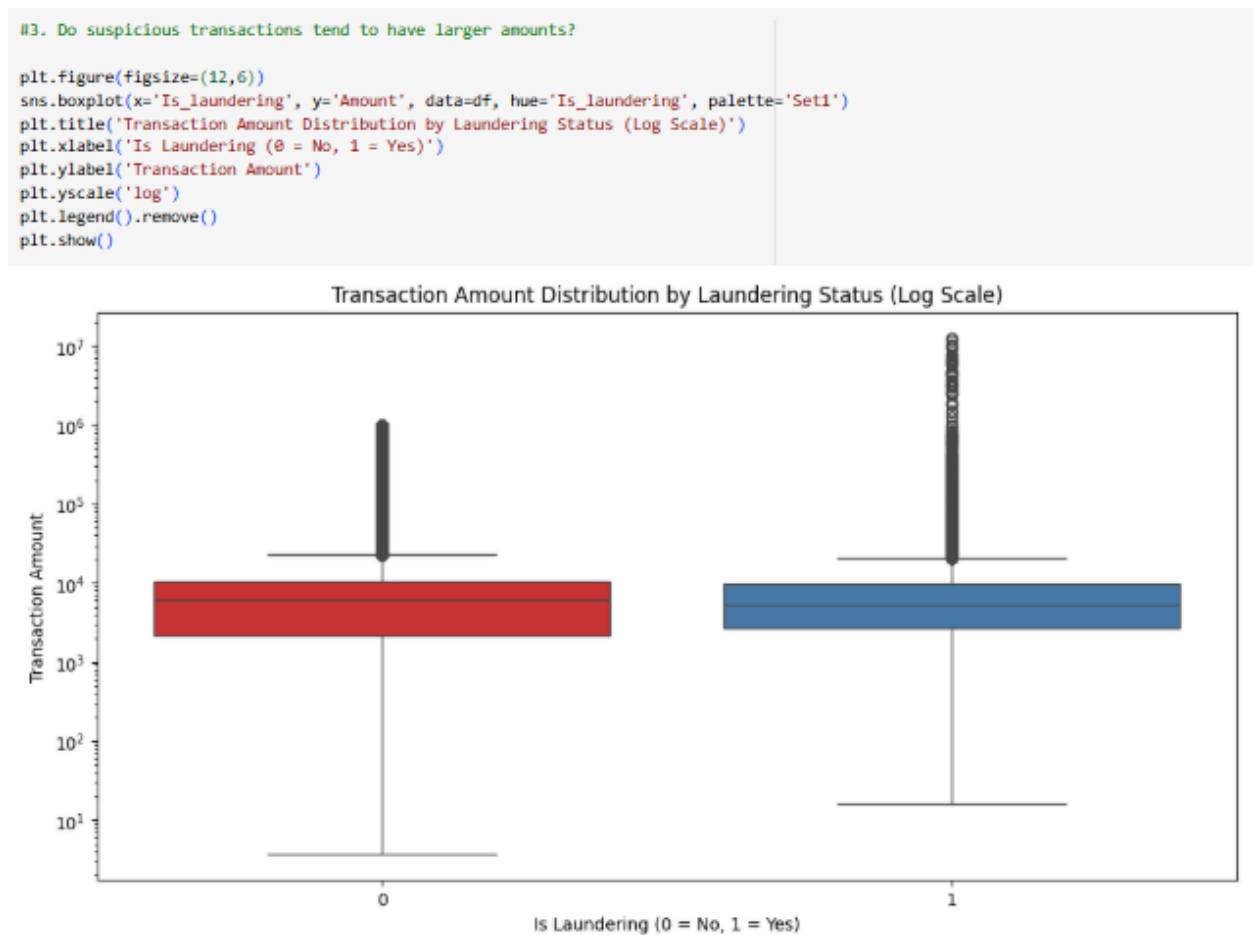


Figure 5.2.3: EDA of the Distribution of Transaction Amounts

Thirdly, the code shows the usage of boxplot to show the distribution of the amount column for both class 1 and class 0 side-by-side for comparison. A log scale is used to manage the skewness in the distribution. The output boxplot visualisations shows that laundering transactions have more extreme outliers on the upper end as laundering cases usually involve the transfer of large sums of funds. But the median value between both the classes are similar which shows that many of the sender accounts involved in laundering transactions also try to fall in the normal value

ranges to avoid detection by thresholds which makes it difficult for authorities to suspect a transaction primarily based on the amount.

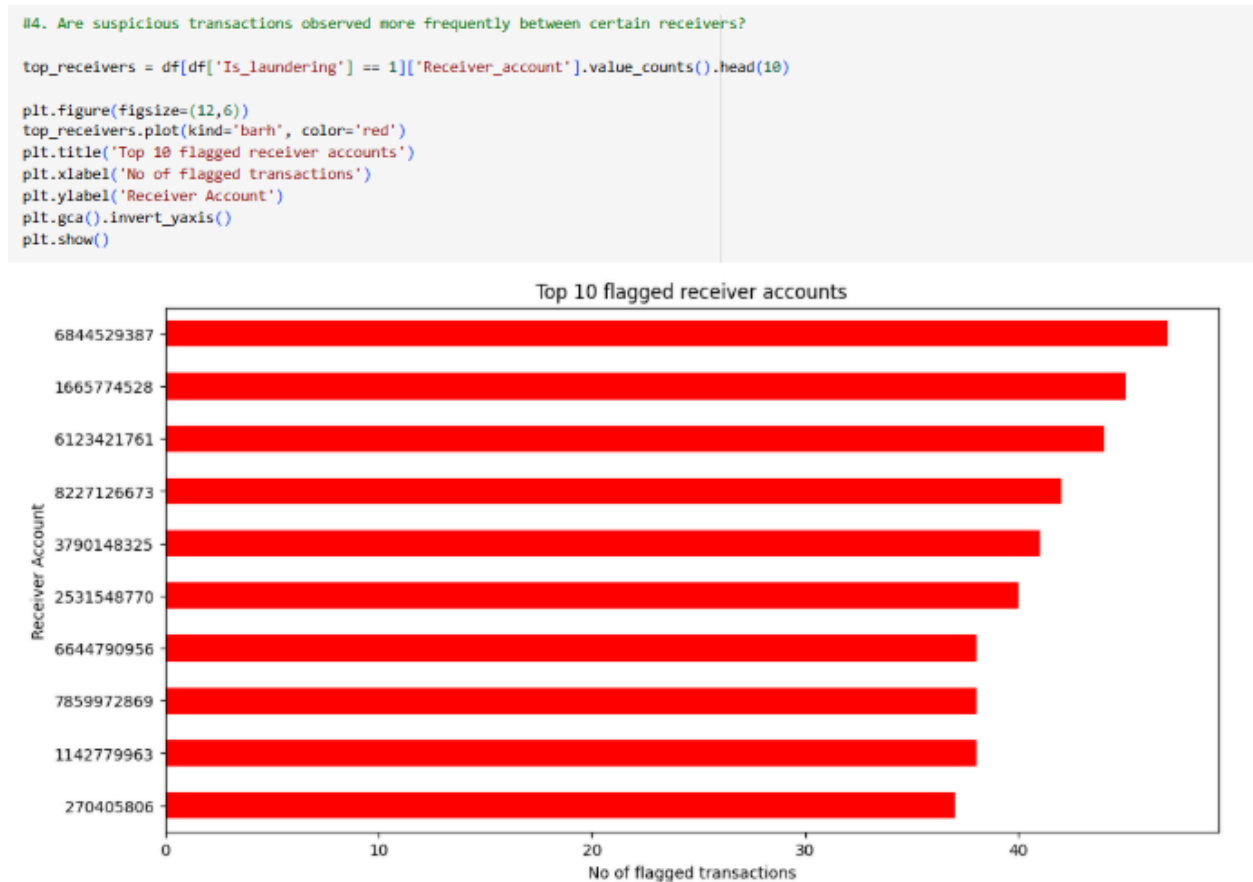


Figure 5.2.4: EDA of the Top Flagged Receiver Accounts

Next, the code used here identifies the Top 10 receiver account numbers that are tied to flagged transactions and uses a horizontal bar chart for visualisation. As seen in Figure 5.2.3, only a small number of receiver accounts are repeatedly flagged in laundering transactions. This means that the laundering networks may be using shell companies or unrelated entities to perform such transactions or they may use money mules or multiple front accounts to avoid being detected for recurring patterns of transactions. Hence, this calls for deeper investigations into how such networks operate their money laundering rings for effective detection.

```
#5. How has the volume of flagged transactions changed over time?

df['DateTime'] = pd.to_datetime(df['Date'].dt.strftime('%Y-%m-%d') + ' ' + df['Time'])
daily_suspicious = df[df['Is_laundering'] == 1].groupby(df['DateTime'].dt.date).size()

plt.figure(figsize=(14,6))
daily_suspicious.plot()
plt.title('Trend of flagged transactions')
plt.xlabel('Date')
plt.ylabel('Freq')
plt.show()
```

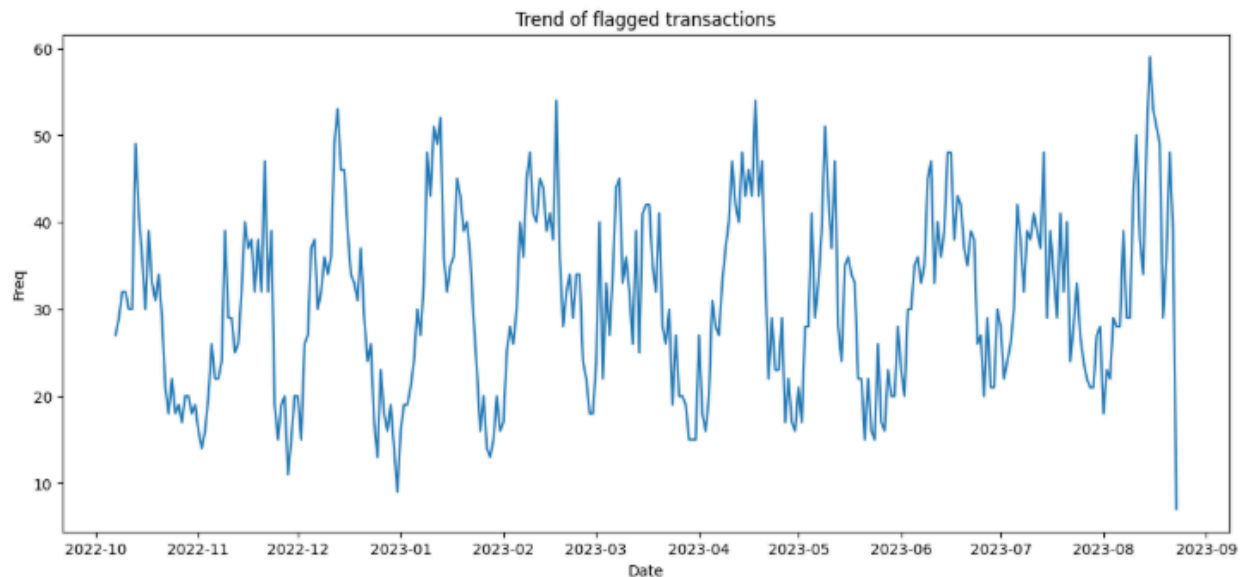


Figure 5.2.5: EDA of the Trend of Flagged Transactions

As the final EDA, this code combines the date and time column into a DateTime field so that the flagged transactions can be aggregated and plotted. Figure 5.2.5 shows the trend output where it can be seen that the number of suspicious transactions highly fluctuate over time. Some visible peaks can be noted in the trend which indicates that it is a significant detection period which could be due to uncovering large scale fraud schemes or bursts of money laundering network strategies. This time based pattern helps authorities detect any regular patterns or periodic intervals between money laundering behaviours and if they are linked to any specific external events like holidays, crypto news and tax reporting deadlines.

6.0 Data Pre-processing and Issues

Several issues were detected in the raw dataset that could affect the performance of the machine learning models we intended to implement. Data preprocessing methods such as random sampling, time-based feature engineering, log transformation for handling skewed data, and one-hot encoding were applied to address these issues and to transform the dataset into a format that can be efficiently processed during machine learning and data mining. Each detected issue and the corresponding preprocessing method used to address it are detailed in this section.

6.1 Random Sampling

```
#Time taken to read the whole dataset
import time
start_time = time.time()

df = pd.read_csv(file_path)
end_time = time.time()
processing_time = end_time - start_time
print(f"Time taken to read the full dataset: {processing_time} seconds")
```

Time taken to read the full dataset: 22.76662540435791 seconds

Figure 6.1.1: Time taken to read the original data set

Figure 6.1.1 above indicates reading the original dataset which includes 9504852 entries takes up to 22 seconds which would affect the efficiency of data modelling in later sections.

```
#Time taken to read only 200,000 rows
start_time = time.time()

#Random sampling of 200000 from the dataset
df = df.sample(n=200000, random_state=1)
end_time = time.time()
processing_time = end_time - start_time
print(f"Time taken to read 200000 rows: {processing_time} seconds")
```

Time taken to read 200000 rows: 0.7063393592834473 seconds

Figure 6.1.2: Time taken to read the random sampling

Random sampling is conducted to reduce the size of data into 200000, allowing us to select a random subset of data that would eliminate biases by giving each distinct data an equal chance of getting chosen which allows us to avoid systematic bias (Utxas.edu, 2020).

6.2 Time-Based Feature Engineering

Time	Date
10:35:19	2022-10-07
10:35:20	2022-10-07
10:35:20	2022-10-07
10:35:21	2022-10-07
10:35:21	2022-10-07

Figure 6.2.1: Original Date and Time format

```
# extract new features From date and time
df['Hour'] = pd.to_datetime(df['Time']).dt.hour

df['Date_Year'] = pd.to_datetime(df['Date']).dt.year
df['Date_Month'] = pd.to_datetime(df['Date']).dt.month
df['Date_Day'] = pd.to_datetime(df['Date']).dt.day

# removing unused columns
df.drop(columns=['Laundering_type', 'Time', 'Date', 'DateTime'], inplace=True)
```

Figure 6.2.2: Time-Based Feature Engineering

Hour	Date_Year	Date_Month	Date_Day
2	2023	8	14
19	2023	7	2
12	2023	7	22
17	2022	10	28
22	2023	2	21

Figure 6.2.3: Date and Time format after Feature Engineering

Time-based feature engineering is applied to transform the datetime data in the dataset into structured features that would enhance model performance. This is particularly important for our model, which aims to predict money laundering activity, where time plays a crucial role. As shown in Figure 6.2.2, we have engineered the original time format (hour:minute:second) by extracting only the hour component to get higher accuracy. Additionally, we extracted date components (year, month, day) as separate features. Redundant columns were then removed to reduce the overall data size.

6.3 Handling Skewed Data

```
# amount has high cardinality - handling skewed data
skewed_data = df['Amount']
original_skewness = skew(skewed_data)
print(f"Original Skewness: {original_skewness}")

log_transformed_data = np.log1p(skewed_data)
transformed_skewness = skew(log_transformed_data)
print(f"Log-Transformed Skewness: {transformed_skewness}")

df['Amount'] = log_transformed_data
```

Original Skewness: 22.99601193607377
Log-Transformed Skewness: -1.0082444862612687

Figure 6.3.1: Skewness of the dataset

The original skewness of the dataset before pre-processing indicates that the dataset is highly skewed which could affect the modelling. Hence, log transform method `[np.log1p (skewed data)]` is performed as the data is positively skewed and this allows us to stabilize variance by reducing the effect of outliers and extreme values.

6.4 One-Hot Encoding

Payment_currency	Received_currency	Sender_bank_location	Receiver_bank_location	Payment_type
UK pounds	UK pounds	UK	UK	Cheque
UK pounds	UK pounds	UK	UK	Debit card
UK pounds	UK pounds	UK	UK	Cheque
UK pounds	Naira	UK	UK	ACH
UK pounds	UK pounds	UK	UK	ACH

Figure 6.4.1: String values of categorical columns

```
# one-hot encoding for categorical columns
categorical_columns = [
    'Payment_currency',
    'Received_currency',
    'Sender_bank_location',
    'Receiver_bank_location',
    'Payment_type'
]
# One-hot encode categorical variables
df = pd.get_dummies(df, columns=categorical_columns, dtype=int)
```

Figure 6.4.2: Code for One-Hot Encoding

[26] df.head()

Payment_currency_Albanian Lek	Payment_currency_Dirham	...	Receiver_bank_location_UAE	Receiver_bank_location_UK	Receiver_bank_location_USA	Payment_type_ACH	Payment_type_Cash Deposit	Payment_type_Cash Withdrawal	Payment_type_Ch
0	0	...	0	1	0	0	0	0	
0	0	...	0	1	0	0	0	0	
0	0	...	0	1	0	0	0	0	
0	0	...	0	1	0	0	0	0	
0	0	...	0	1	0	0	0	0	

Figure 6.4.3: Output of the code

Categorical columns that were mentioned above have string values which would make it harder for models to understand. One-hot encoding is implemented to convert the string values into numerical format by using ‘dtype=int’ and convert each category in the original column to its own column by using ‘df = pd.get_dummies(df, columns=categorical_columns)’. This results in

the increment of the number of columns, increasing dimensionality and each category is treated independently for the model.

7.0 Data Science Techniques

Classification is a supervised machine learning algorithm whose goal is to make predictions on the class labels for input data provided based on learned patterns. The following are the steps taken to implement XGBoost and Random Forest Classifier models for the SAML-D dataset.

7.1 XGBoost

The reason for using XGBoost is because it is appropriate for structured data and classification issues such as imbalance datasets. This is because it possesses parameters like “scale_pos_weight” and offers strong regularisation to avoid overfitting. It works by creating a collection of decision trees in a sequence such that the new tree corrects the mistakes of the preceding trees. This approach helps to improve performance by focusing on examples that are harder to classify and works well with large datasets.

Step 1: Identifying the target and dependent variables

The dependent and target variables are represented by X and y. The target variable is ‘Is_laundering’ because it shows whether money laundering happens or not.

```
X = df1.drop(columns=['Is_laundering'])  
y = df1['Is_laundering']
```

Figure 7.1.1: Identifying Target and Dependent Variables

Step 2: Handling class imbalance

SMOTE sampling technique method is used to handle class imbalance in the dataset. By using the fit_resample(), it generates synthetic samples for the minority class which is laundering transactions so that both classes have an equal number of records.

```
# apply resampling technique using SMOTE
smote = SMOTE(random_state=42)

X_resampled, y_resampled = smote.fit_resample(X, y)
```

Figure 7.1.2: Applying Smote Technique

After resampling, a new DataFrame is created with the resampled features as 'X_resampled' and target variable labels as 'y_resampled'.

```
data = pd.DataFrame(X_resampled, columns=X.columns)
data['Is_laundering'] = y_resampled
```

Figure 7.1.3: DataFrame for Resampled Data

The outcome of the sampling method ensures both classes receive 50% of the dataset each. This means the class distribution is balanced. As a result, the model can learn patterns from the two classes effectively and not be biased towards the majority class.

```
print("New class distribution:")
print(data['Is_laundering'].value_counts(normalize=True))
```

New class distribution:
Is_laundering
0 0.5
1 0.5
Name: proportion, dtype: float64

Figure 7.1.4: Result of Smote Sampling Technique

Step 3: Dataset Splitting

The balanced data is split into training, validation and test sets. 10% is for the testing set, another 10% is for the validation set, and the remaining 80% for the training set using the 'train_test_split()' function. The 'stratify' parameter is used so that class proportions remain balanced in all the splits. This allows the model to train, tune, and test on different data,

preventing overfitting and allowing generalisation to new data more effectively. After splitting, the sizes of the training, validation and testing sets are as follows:

```
X_temp, X_test, y_temp, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.1, stratify=y_resampled, random_state=42
)
X_train, X_validation, y_train, y_validation = train_test_split(
    X_temp, y_temp, test_size=0.111, stratify=y_temp, random_state=42
)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)

(319727, 76) (319727,) (39961, 76) (39961,)
```

Figure 7.1.5: Splitting the Balanced Dataset

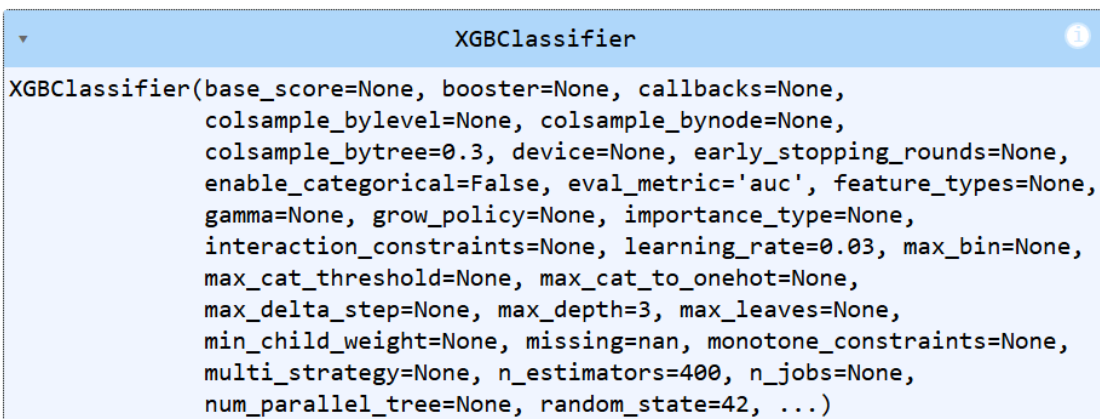
Step 4: Training XGBoost

In this step, XGBoost is trained using the ‘fit()’ function on the SMOTE-generated balanced dataset. The model is tuned using the following hyperparameters:

- **eval_metric='auc':** Model performance is measured by AUC that is appropriate for binary classification.
- **random_state=42:** Produces deterministic results on multiple runs which is essential in comparing model performance after hyperparameter tuning.
- **max_depth=3:** Prevents overfitting by limiting the depth of the trees and performs well with high-dimensional data after one-hot encoding
- **learning_rate=0.03:** It regulates the learning rate of the model. The model learns gradually with a lower value and does not overfit.
- **n_estimators=400:** It indicates the number of trees. More trees together with the low learning rate, the model is more stable.
- **colsample_bytree=0.3:** 80% of the features by each tree so that the model does not rely on one feature.
- **subsample=0.8:** Chooses 80% of the training data randomly for each tree to enhance generalisation and accelerate training on a huge dataset

- **reg_alpha=0.5:** Implements L1 regularization to cause sparsity such as making some feature weights to zero and prevent overfitting.
- **reg_lambda=1:** Implements L2 regularization to shrink large weights to make the model less responsive to noise and correlated features.

```
xgb = XGBClassifier(  
    eval_metric='auc',  
    random_state=42,  
    max_depth=3,  
    learning_rate=0.03,  
    n_estimators=400,  
    colsample_bytree=0.3,  
    subsample=0.8,  
    reg_alpha=0.5,  
    reg_lambda=1,  
)  
  
xgb.fit(X_resampled, y_resampled)
```



The screenshot shows a Jupyter Notebook cell with the title 'XGBClassifier'. Below the title, the full list of attributes and their values for the XGBClassifier object is displayed. The attributes include base_score, booster, callbacks, colsample_bylevel, colsample_bynode, colsample_bytree, device, early_stopping_rounds, enable_categorical, eval_metric, feature_types, gamma, grow_policy, importance_type, interaction_constraints, learning_rate, max_bin, max_cat_threshold, max_cat_to_onehot, max_delta_step, max_depth, max_leaves, min_child_weight, missing, monotone_constraints, multi_strategy, n_estimators, n_jobs, num_parallel_tree, random_state, and many others. The values are mostly None, with some specific values like 0.3, 0.03, 400, 0.8, 0.5, and 1.0.

```
XGBClassifier(base_score=None, booster=None, callbacks=None,  
             colsample_bylevel=None, colsample_bynode=None,  
             colsample_bytree=0.3, device=None, early_stopping_rounds=None,  
             enable_categorical=False, eval_metric='auc', feature_types=None,  
             gamma=None, grow_policy=None, importance_type=None,  
             interaction_constraints=None, learning_rate=0.03, max_bin=None,  
             max_cat_threshold=None, max_cat_to_onehot=None,  
             max_delta_step=None, max_depth=3, max_leaves=None,  
             min_child_weight=None, missing=nan, monotone_constraints=None,  
             multi_strategy=None, n_estimators=400, n_jobs=None,  
             num_parallel_tree=None, random_state=42, ...)
```

Figure 7.1.6: Fitting XGBoost Model

7.2 Random Forest

Random Forest is a machine learning algorithm that is selected in this project. This is because this algorithm can make a superior prediction by using a large number of decision trees, each of which will examine distinct random segments of the data, and their result will be combined by voting classification. This approach has assisted in increasing the accuracy as well as reducing the error.

Step 1: Model Training

```
[31] # Train a Model (Random Forest Classifier)
      rf = RandomForestClassifier(n_estimators=100,max_features=0.3,max_depth=5,random_state=42)
      rf.fit(X_resampled, y_resampled)
```

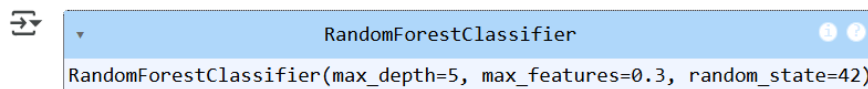


Figure 7.2.1: Fitting Random Forest Model

In this step, the Random Forest algorithm will be applied to the dataset. Random Forest Classifier is initialised with specific parameters.

Where,

- **n_estimators = 100:** indicates that there are 100 decision trees in the forest; more trees will suggest a better performance, but with a relatively longer execution time.
- **max-depth = 5:** Restricts the depth of the tree to just 5 in order to prevent overfitting issues..
- **max_features = 0.3:** 30% of the features will be selected at random for each tree when splitting a node.
- **random_state = 42:** To ensure that each time to code is executed, the same result will be produced.

Step 2: Predict the Test Dataset

```
[ ] #Predict test dataset  
    y_pred = rf.predict(X_test)
```

Figure 7.2.2: Testing dataset

The trained random forest will then be utilised to make predictions on the dataset in order to evaluate the performance of the random forest model.

8.0 Model Validation

The models are validated and evaluated in terms of three aspects, which are classification report, Receiver Operating Characteristic (ROC) curve, and confusion matrix.

8.1 XGBoost

Classification Report

The block of code starts by generating class probabilities for validation and test datasets using the 'predict_proba()' function. The class probabilities are used to calculate the ROC AUC scores. Next, prediction is done using the 'predict()' function to get class labels. The class labels are for performance metrics such as precision, recall, and F1-score for both classes. Precision measures how many of the predicted laundering transactions were indeed laundering, recall measures how many of the actual laundering transactions were predicted correctly, and F1-score balances between the two by measuring false negatives in addition to false positives.

```
val_probs = xgb.predict_proba(X_validation)[: , 1]
test_probs = xgb.predict_proba(X_test)[: , 1]

val_auc = roc_auc_score(y_validation, val_probs)
test_auc = roc_auc_score(y_test, test_probs)

val_preds = xgb.predict(X_validation)
test_preds = (test_probs >= 0.2).astype(int)
train_preds = xgb.predict(X_train)

print("Validation AUC:", val_auc)
print("Test AUC:", test_auc)

print("\nValidation Report:\n", classification_report(y_validation, val_preds, zero_division=0))
print("Validation Accuracy:", accuracy_score(y_validation, val_preds))

print("\nTest Report:\n", classification_report(y_test, test_preds, zero_division=0))
print("Training Accuracy:", accuracy_score(y_train, train_preds))
print("Test Accuracy:", accuracy_score(y_test, test_preds))
```

Figure 8.1.1: Classification Report for XGBoost

The classification report illustrates excellent model performance on all data sets. AUC values are extremely high, Validation AUC = 0.99 and Test AUC = 0.99, showing the excellent differentiation ability of the model between laundering and non-laundering transactions.

Precision, recall, and F1-measures are high (approximately 0.87-0.99) for both classes, showing that the model distinguishes laundering cases extremely well with very few false positives or false negatives. In addition to that, the training accuracy percentages (96.66%), validation (96.57%) and test (92.80%) are very close to each other, which suggests that the model is generalising well and not overfitting.

```

Validation AUC: 0.9946933417112419
Test AUC: 0.9946759105835229

Validation Report:
      precision    recall  f1-score   support

     0       0.95      0.98      0.97     19961
     1       0.98      0.95      0.97     19961

 accuracy          0.97          0.97          0.97     39922
 macro avg       0.97      0.97      0.97     39922
 weighted avg    0.97      0.97      0.97     39922

Validation Accuracy: 0.9657331797004158

Test Report:
      precision    recall  f1-score   support

     0       0.99      0.87      0.92     19981
     1       0.88      0.99      0.93     19980

 accuracy          0.93          0.93          0.93     39961
 macro avg       0.93      0.93      0.93     39961
 weighted avg    0.93      0.93      0.93     39961

Training Accuracy: 0.9665933749730239
Test Accuracy: 0.9280298290833563

```

Figure 8.1.2: Result of Classification Report for XGBoost

Confusion Matrix

The confusion matrix for XGBoost is plotted using the 'confusion_matrix()' function.

```
cm = confusion_matrix(y_test, test_preds)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix (Test Set)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

Figure 8.1.3: Confusion Matrix for XGBoost

Figure 8.1.4 shows the result of the confusion matrix, the model correctly identified 19754 laundering cases and 17331 non-laundering cases. It only made 2650 false positives which non-laundering cases were predicted as laundering and 226 false negatives which laundering cases were predicted as non-laundering. This indicates the model is highly effective at identifying both classes.

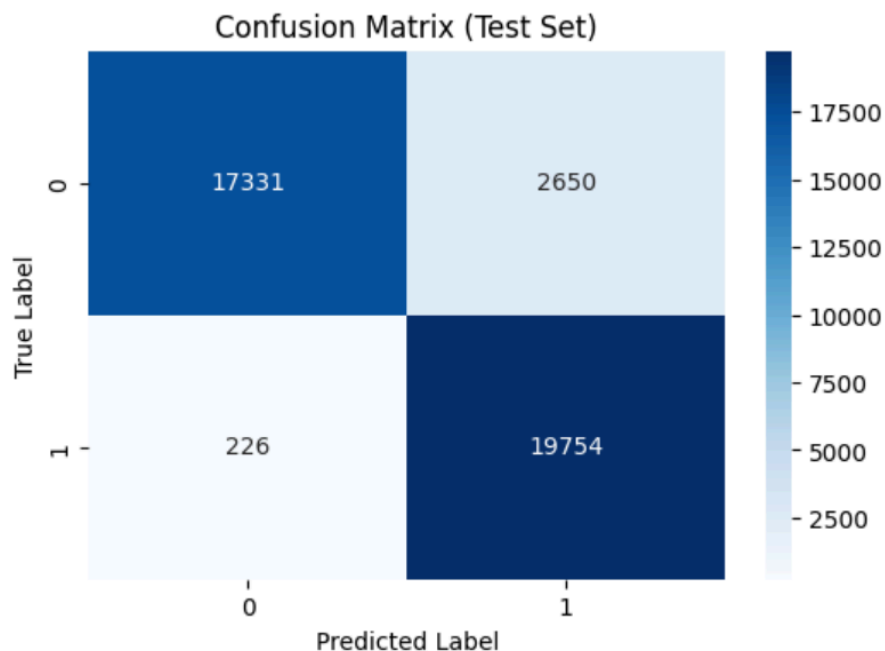


Figure 8.1.4: Results of Confusion Matrix for XGBoost

Receiver Operating Characteristic (ROC) curve

The block of code is to plot the ROC curve of the test set using the `roc_curve()` function. The ROC curve computes the model's capability in differentiating between the positive and negative classes. It starts with computing the false positive rate (FPR) and true positive rate (TPR) at different threshold levels using the `roc_curve()` function. The values are plotted where the orange line represents the performance of the model.

```
fpr, tpr, thresholds = roc_curve(y_test, test_probs)
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {test_auc:.2f})", color='darkorange')
plt.plot([0, 1], [0, 1], linestyle='--', color='navy')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (Test Set)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

Figure 8.1.5: Receiver Operating Characteristic (ROC) for XGBoost

The ROC curve plotted has a steep incline and remains close to the upper left corner which means that the model is correctly identifying most positives and maintaining low false positives. The AUC score is 0.99 which implies perfect classification performance on the test set. This indicates the model is extremely good at differentiating between laundering and non-laundering transactions. The high score indicates that the model has picked up very good patterns from the data.

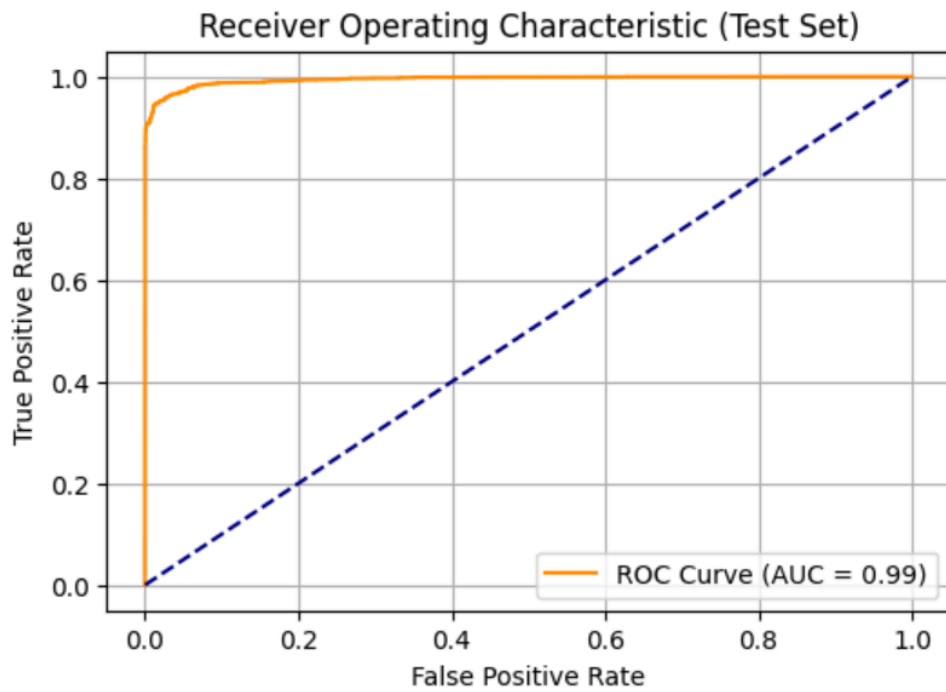


Figure 8.1.6: Result of Receiver Operating Characteristic (ROC) for XGBoost

8.2 Random Forest

Classification Report

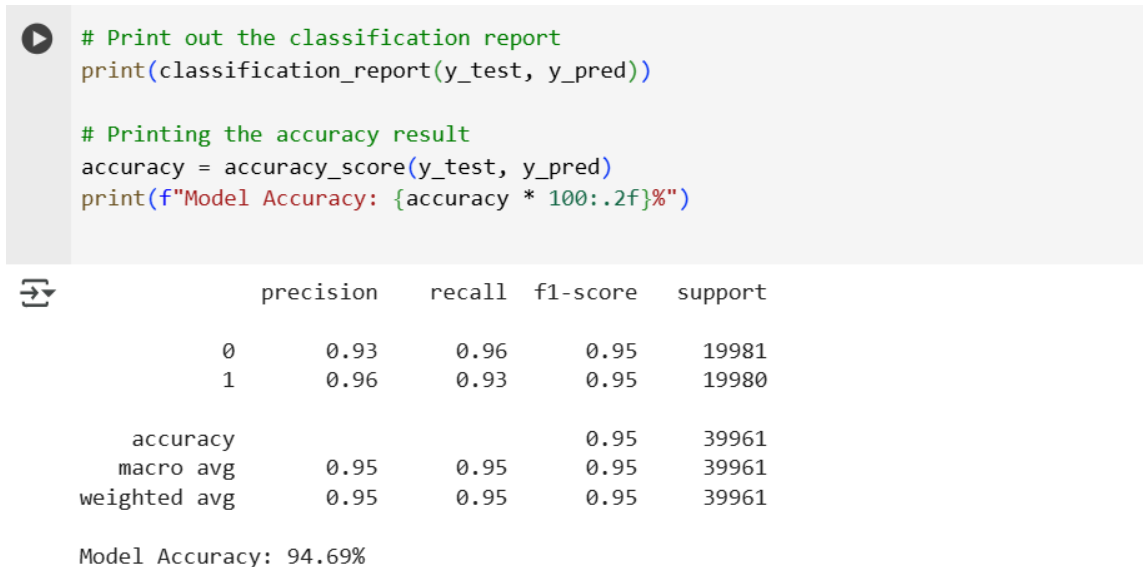


Figure 8.2.1: Result of Classification Report for Random Forest

The classification report above for the random forest model has demonstrated an incredible performance of the model by reaching an accuracy of up to 95%. Based on the report, the report consists of two classes, class 0 and class 1, and both of them contain nearly equal representation (19981 and 19980). This suggests that the model is good for evaluation. Besides that, the precision for both classes, which is 93% and 96%, suggests that the model can correctly predict the classes with minimal false rate. Furthermore, the recall score for both classes suggests that the model can accurately identify 96% of actual non-laundering activities and 93% of laundering activities. Lastly, the high value of f1-score for both classes (95%) has also indicated that the model is performing well.

Confusion Matrix

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

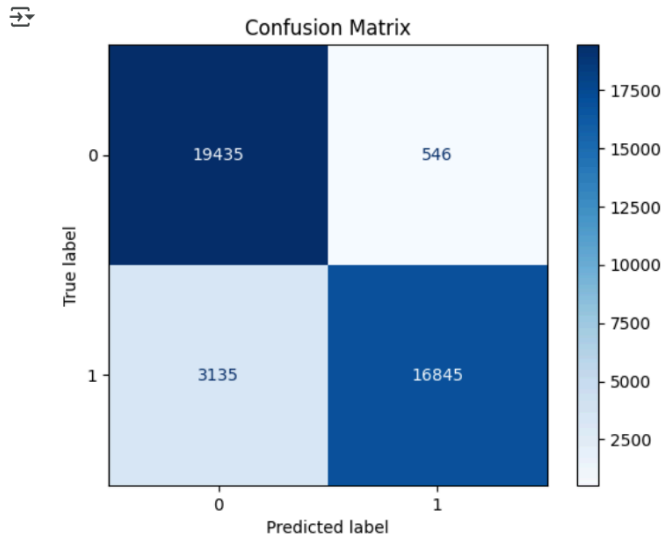


Figure 8.2.2: Results of the Confusion Matrix for Random Forest

The result of the confusion matrix suggests that the model can accurately identify 19,435 true positives and 16,845 true negatives, and only identify 546 negative cases as positive and 3135 positive cases as negative. This suggests that the model can effectively classify both classes in an accurate manner.

Receiver Operating Characteristic (ROC) curve

```
[38] # ROC Curve
y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
# Plot ROC curve for the best model
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"Best Model (AUC={roc_auc:.2f})", color='blue')
# Baseline (Random Classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels & Legends
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve for Best Random Forest Model')
plt.legend()
plt.grid()
plt.show()
```

Figure 8.2.3: Code of ROC Curve

The above block of code is to visualise the ROC curve in order to illustrate the trade-off between the true positive rate (TPR) and false positive rate (FPR).

Output:

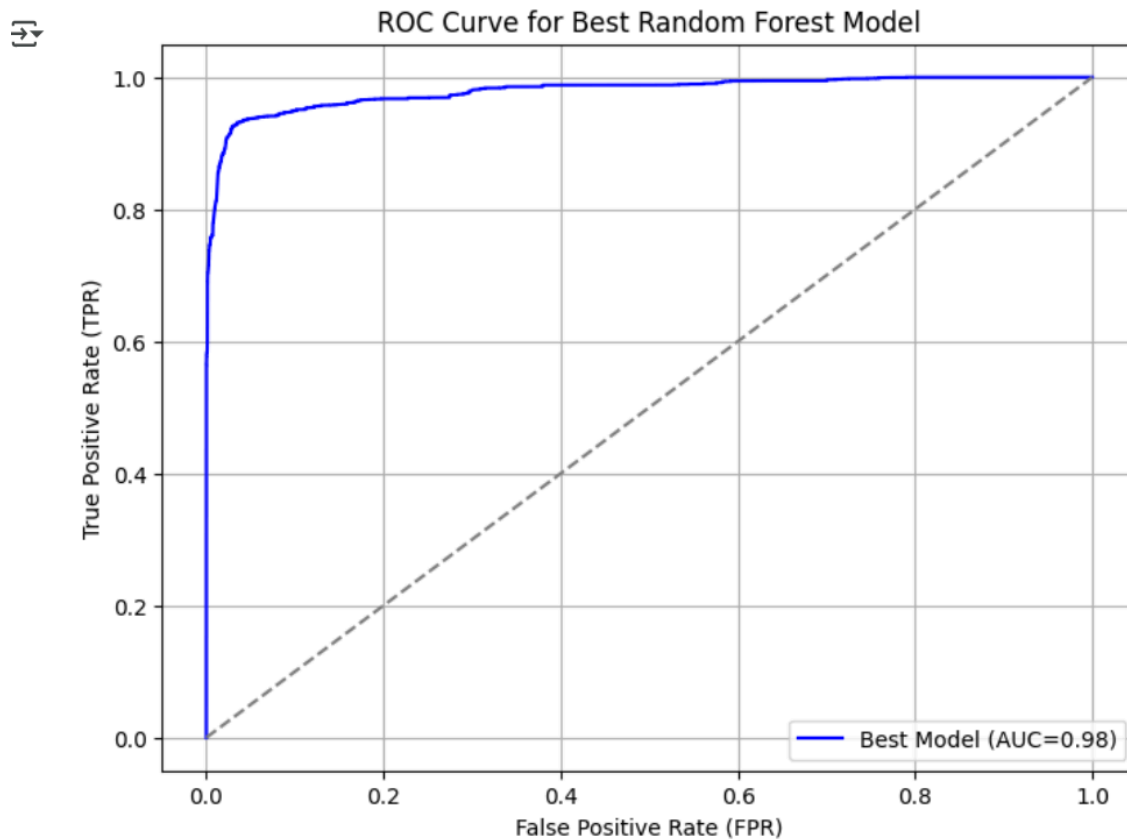


Figure 8.2.4: Result of ROC curve for Random Forest

Based on the ROC curve for random forest model, the line, which is approaching the top-left corner, has suggested a good performance of the model, with the Area Under Curve (AUC) value achieved up to 98%. This has indicated a discriminatory power of the model, by correctly classifying both positive cases (money laundering) and negative cases (non-money laundering) with minimal error.

9.0 Conclusion

In this project, the SAML-D dataset has been explored to identify potential money laundering activities using various data science techniques, including data preprocessing, exploratory data analysis, and machine learning algorithms. According to the observation, the target class, “is_Laundering”, is highly imbalanced, with the suspicious transaction instances comprising less than 0.1%. Therefore, to address the severe class imbalance issues, the resampling technique SMOTE has been applied to balance the dataset by increasing the sample size of the minority class. In addition, a significant limitation encountered in this project was the large dataset size that contains more than 9 millions of records. This has impacted some preprocessing steps like encoding and scaling, and even model training and evaluation, resulting in long processing time and Google Colab crash due to large RAM usage. To solve this problem, a sample of 200,000 records was utilised to build the model while retaining data diversity.

Furthermore, two machine learning models, including XGBoost Classifier and Random Forest Classifier, have been assessed, where for XGBoost Classifier, the accuracy can reach up to 97% of accuracy with an AUC value of 99%, as for Random Forest Classifier, the accuracy can achieve 95% with a 98% AUC value. The result of both models indicates that the performance of the XGBoost Classifier is better than Random Forest Classifier. This is because the XBoost is designed for handling large-scale data efficiently by sequentially improving upon the error made by previous trees.

Despite the present model achieving a promising result in identifying money laundering activities, there are a number of areas that might be explored to improve its efficacy and practicality, including apply a more advanced algorithm, such as a deep learning model in order to reveal a more complex pattern in high-dimensional data, integrate with Artificial Intelligence techniques to increase the model's transparency and accuracy and incorporate additional features to enhance the model's capacity to identify a more complex scheme in money laundering, such as a network-based transaction pattern.

10.0 Colab Notebook

The Google Colab file can be accessed through the link below:

 [DSP_Group 18_Money Laundering Detection](#)

11.0 References

1. Crossman-Smith, J. (2024, November 29). *XGBoost explained: A beginner's guide - low code for data science - medium*. Medium.
<https://medium.com/low-code-for-advanced-data-science/xgboost-explained-a-beginners-guide-095464ad418f>
2. Flagright. (2024, May 17). *Ensuring data accuracy in AML systems*.
<https://www.flagright.com/post/ensuring-data-accuracy-in-aml-systems>
3. Ross, M. L. (2023, September 19). *HSBC's Money Laundering Scandal*. Investopedia.
<https://www.investopedia.com/stock-analysis/2013/investing-news-for-jan-29-hsbcs-money-laundering-scandal-hbc-scbff-ing-cs-rbs0129.aspx#:~:text=HSBC%27s%202012%20scandal%20served%20as%20a%20watershed%20moment,violations%20suggest%20there%27s%20more%20work%20to%20be%20done.>
4. Utxas.edu. (2020). *Why Is Random Sampling Important?* [online] Available at:
<https://web.ma.utexas.edu/users/mks/statmistakes/RandomSampleImportance.html>.
5. United Nations (2024). *Money laundering*. [online] United Nations : Office on Drugs and Crime. Available at: <https://www.unodc.org/unodc/en/money-laundering/overview.html>.
6. CFI Team (2015.). *Money Laundering*. [online] Corporate Finance Institute. Available at:
<https://corporatefinanceinstitute.com/resources/career-map/sell-side/risk-management/money-laundering/>.
7. Napier AI (2020). *Look before you leap: the risks and challenges of implementing AI for AML*. [online] Available at: <https://www.napier.ai/post/ai-aml-challenges>.
8. Financial Crime Academy. (2024). *Safeguarding Against Money Laundering: Essential Machine Learning Algorithms For AML*. [online] Available at:
<https://financialcrimeacademy.org/machine-learning-algorithms-for-aml/>.
9. Xiao, J. (2024). *The Application of Machine Learning in Financial Fraud*.
<https://doi.org/10.36227/techrxiv.172841690.07508504/v2>
10. AL-Dahasi, E. M., Alsheikh, R. K., Khan, F. A., & Jeon, G. (2024). Optimizing fraud detection in financial transactions with machine learning and imbalance mitigation. *Expert Systems*. <https://doi.org/10.1111/exsy.13682>
11. Hossain, M. Z., Raja, M. R., & Hasan, L. (2024). Developing Predictive Models for Detecting Financial Statement Fraud: A Machine Learning Approach. *European Journal*

of Theoretical and Applied Sciences, 2(6), 271–290.
[https://doi.org/10.59324/ejtas.2024.2\(6\).22](https://doi.org/10.59324/ejtas.2024.2(6).22)

12. Pandey, N., Singh, S., & Singh, A. (2024). *Online Transaction Fraud Detection Using Deep Learning*. 1–6. <https://doi.org/10.1109/icicet59348.2024.10616357>
13. Schickler, J. (2023, May 5). UK's FCA Continues Crackdown on Unregistered Crypto ATMs. *CoinDesk*.
<https://www.coindesk.com/policy/2023/05/05/uks-fca-continues-crackdown-on-unregistered-crypto-atms>
14. O'Donoghue, P. (2025, March 10). *BREAKING: Four convicted in £266M money laundering case, believed to be biggest ever in UK*. AML Intelligence.
<https://www.amlintelligence.com/2025/03/breaking-four-convicted-in-266m-money-laundering-case-believed-to-be-biggest-ever-in-uk/#:~:text=FOUR%20men%20have%20been%20found%20guilty%20of%20laundering,money%20laundering%20case%20ever%20prosecuted%20in%20the%20UK>.
15. Oztas, B., Cetinkaya, D., Adedoyin, F., Budka, M., Dogan, H., & Aksu, G. (2023). Enhancing Anti-Money Laundering: Development of a Synthetic Transaction Monitoring Dataset. *2023 IEEE International Conference on e-Business Engineering (ICEBE)*, 47–54. <https://doi.org/10.1109/icebe59045.2023.00028>